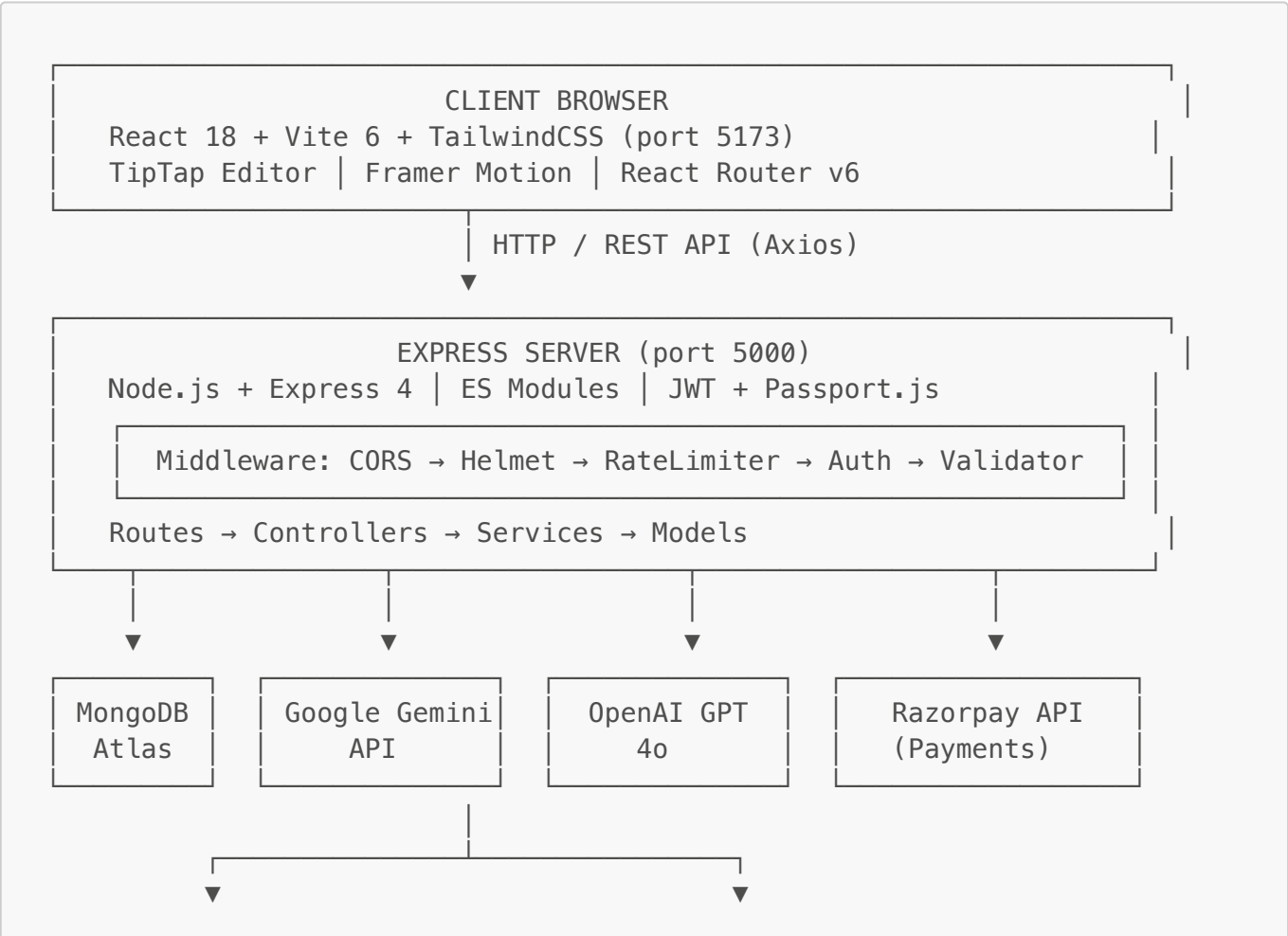


SmartNShine ATS Resume Generator — System Design

Table of Contents

- 1. [Architecture Overview](#)
- 2. [Frontend Architecture](#)
- 3. [Backend Architecture](#)
- 4. [Database Design](#)
- 5. [Authentication & Authorization](#)
- 6. [Core Feature Flows](#)
- 7. [Voice Pipeline](#)
- 8. [Payment Flow](#)
- 9. [AI Integration Layer](#)
- 10. [Middleware Stack](#)
- 11. [Admin Architecture](#)
- 12. [External Services](#)
- 13. [Rate Limiting & Quota System](#)

1. Architecture Overview



Whisper STT Service
Python Flask :5001
OpenAI Whisper

Chatterbox TTS Service
Python Flask :5002
Chatterbox Model

Key Design Principles

- **MVC + Service Layer:** Routes → Controllers → Services → Models
- **ES Modules** throughout (import/export, no require)
- **JWT Stateless Auth** with Passport.js for OAuth
- **Microservice separation** for AI voice (Python Flask services)
- **Quota-per-tier** subscription model for all AI features

2. Frontend Architecture

2.1 Client-Side Routes

```
App.jsx (React Router v6)
├── Layout (Navbar + Footer wrapper)
│   ├── / → Home
│   ├── /login → Login
│   ├── /register → Register
│   ├── /forgot-password → ForgotPassword
│   ├── /reset-password → ResetPassword
│   ├── /auth/callback → OAuthCallback
│   ├── /dashboard → Dashboard (Protected)
│   ├── /upload → UploadResume (Protected)
│   ├── /editor/:id? → Editor (Protected)
│   ├── /templates → Templates (Protected)
│   ├── /ats-analyzer → ATSanalyzer (Protected)
│   ├── /pricing → Pricing
│   ├── /subscription-dashboard → SubscriptionDashboard (Protected)
│   ├── /profile → Profile (Protected)
│   ├── /contact → Contact (Protected)
│   ├── /smart-job-match → SmartJobMatch (Protected)
│   ├── /ai-interview → AIInterview (Protected)
│   ├── /interview-history → InterviewHistory (Protected)
│   ├── /interview-result/:id → InterviewResult (Protected)
│   ├── /github-import → GitHubImport (Protected)
│   ├── /advanced-analytics → AdvancedAnalytics (Protected)
│   ├── /privacy-policy → PrivacyPolicy
│   ├── /terms → Terms
│   ├── /refund-policy → RefundPolicy
│   ├── /shipping-policy → ShippingPolicy
│   └── /* → NotFound
└── AdminLayout (Admin wrapper)
    ├── AdminProtectedRoute (role: "admin")
    │   ├── /admin/dashboard → AdminDashboard
```

```

├── /admin/users           → AdminUsers
├── /admin/ai-analytics    → AdminAIAalytics
├── /admin/ai-quota        → AdminAIQuota
├── /admin/ai-extraction   → AdminAIExtraction
├── /admin/contacts        → AdminContacts
├── /admin/feedback        → AdminFeedback
├── /admin/logs            → AdminLogs
├── /admin/templates       → AdminTemplates
├── /admin/settings        → AdminSettings
├── /admin/earnings        → AdminEarnings

```

2.2 Component Hierarchy

```

src/
├── pages/
│   ├── AIInterview.jsx      (3300+ lines – main interview UI)
│   ├── Editor.jsx          (Resume WYSIWYG editor)
│   ├── Dashboard.jsx
│   ├── ATSAAnalyzer.jsx
│   ├── InterviewResult.jsx
│   └── ...
├── components/
│   ├── layout/
│   │   ├── Navbar.jsx
│   │   ├── Footer.jsx
│   │   └── Layout.jsx
│   ├── editor/
│   │   ├── ResumeEditor.jsx  (TipTap editor wrapper)
│   │   ├── SectionEditor.jsx
│   │   └── FloatingActionBar.jsx
│   ├── templates/
│   │   ├── ClassicTemplate.jsx
│   │   ├── ModernTemplate.jsx
│   │   ├── MinimalTemplate.jsx
│   │   └── ... (maxHeight: 1056px for all)
│   ├── admin/
│   │   ├── AdminLayout.jsx
│   │   ├── AdminProtectedRoute.jsx
│   │   └── ... (analytics, user mgmt panels)
│   ├── auth/
│   │   ├── ProtectedRoute.jsx
│   │   └── OAuthButtons.jsx
│   └── common/
│       ├── LoadingSpinner.jsx
│       ├── Modal.jsx
│       └── ...
├── api/
│   ├── api.js              (base Axios instance → :5000)
│   └── interview.api.js    (interview endpoints)

```

```
├── voice.api.js           (voice endpoints)
│   └── ...
├── context/
│   ├── DarkModeContext.jsx
│   └── NavigationBlockerContext.jsx
├── hooks/
│   ├── useInterview.js
│   ├── useVoice.js
│   └── ...
```

2.3 State Management

- **No Redux** — React Context + local useState/useReducer
- **DarkModeContext** — global theme toggle
- **NavigationBlockerContext** — unsaved changes guard
- Interview state managed locally in **AIInterview.jsx** (~3300 lines)

3. Backend Architecture

3.1 Server Entry Point

```
server.js
├── Express app setup
├── Middleware chain (CORS, Helmet, JSON parser, session)
├── Passport initialization (Google OAuth, GitHub OAuth)
├── Route mounting:
│   ├── /api/auth           → auth.routes.js
│   ├── /api/resume         → resume.routes.js
│   ├── /api/ats            → ats.routes.js
│   ├── /api/interview      → interview.routes.js
│   ├── /api/voice          → voice.routes.js
│   ├── /api/subscription  → subscription.routes.js
│   ├── /api/admin          → admin.routes.js
│   ├── /api/contact        → contact.routes.js
│   ├── /api/feedback       → feedback.routes.js
│   └── /api/github         → github.routes.js
└── MongoDB Atlas connection (Mongoose)
```

3.2 API Routes Summary

Authentication **/api/auth**

Method	Path	Purpose	Auth
POST	/register	User registration	No

Method	Path	Purpose	Auth
POST	/login	User login → JWT	No
POST	/forgot-password	Request reset email	No
POST	/reset-password	Reset with token	No
GET	/google	Google OAuth initiate	No
GET	/google/callback	Google OAuth callback	No
GET	/github	GitHub OAuth initiate	No
GET	/github/callback	GitHub OAuth callback	No
GET	/me	Get current user	Yes
GET	/oauth-status	OAuth config check	No

Resume /api/resume

Method	Path	Purpose	Auth
POST	/upload	Upload PDF/DOCX + AI parse	Yes
POST	/enhance	AI-enhance resume section	Yes
POST	/generate-summary	AI generate summary	Yes
POST	/categorize-skills	AI categorize skills	Yes
POST	/segregate-achievements	AI segregate achievements	Yes
POST	/process-custom-section	AI process custom section	Yes
POST	/save	Save resume to DB	Yes
POST	/track-download	Track download event	Yes
PUT	/:id	Update resume	Yes
GET	/list	List user resumes	Yes
GET	/:id	Get resume by ID	Yes
DELETE	/:id	Delete resume	Yes

ATS Analysis /api/ats

Method	Path	Purpose
POST	/analyze-resume	Full ATS analysis vs job desc
POST	/match-score	Calculate keyword match %
POST	/analyze-skills	Skills gap analysis

AI Interview `/api/interview`

Method	Path	Purpose
GET	<code>/config</code>	Interview config options
POST	<code>/sessions</code>	Create session
POST	<code>/sessions/:id/start</code>	Start, get first question
POST	<code>/sessions/:id/answer</code>	Submit text answer
POST	<code>/sessions/:id/voice-answer</code>	Submit audio (Whisper STT)
POST	<code>/sessions/:id/skip</code>	Skip question
POST	<code>/sessions/:id/complete</code>	Complete + generate report
POST	<code>/sessions/:id/abandon</code>	Abandon session
GET	<code>/sessions/:id</code>	Get session details
GET	<code>/results/:id</code>	Get final report
GET	<code>/history</code>	Interview history
GET	<code>/stats</code>	Interview statistics

Voice `/api/voice`

Method	Path	Purpose
GET	<code>/health</code>	Whisper STT health check
GET	<code>/tts/health</code>	Chatterbox TTS health check
GET	<code>/transcribe/health</code>	Transcription capability check
POST	<code>/tts/synthesize</code>	Text → Audio (Chatterbox → Browser fallback)

Subscription `/api/subscription`

Method	Path	Purpose
GET	<code>/pricing</code>	Pricing info (public)
POST	<code>/create-order</code>	Create Razorpay order
POST	<code>/verify-payment</code>	Verify + activate subscription
POST	<code>/cancel</code>	Cancel subscription
POST	<code>/renew</code>	Renew subscription
GET	<code>/status</code>	Current subscription status
GET	<code>/usage</code>	Usage stats

Method	Path	Purpose
POST	/webhook	Razorpay webhook (no auth)

Admin /api/admin (40+ endpoints, role: admin)

All require: `authenticateToken` + `isAdmin` + `adminLimiter` + `logAdminAction`

- **Dashboard:** /dashboard/stats
- **Users:** CRUD + role/status management
- **AI Quota:** Per-user quota management, tier updates, resets
- **AI Analytics:** Usage analytics, extraction stats
- **Contacts:** View, update status, delete
- **Feedback:** View, moderate, delete
- **Settings:** System config, feature flags, rate limits
- **Logs:** Admin action audit trail
- **Templates:** Template management
- **Earnings:** Payment analytics

4. Database Design

4.1 Entity Relationship



4.2 User Schema

```
User {
  // Auth
  email: String (unique)
  password: String (bcrypt hashed)
  name: String
  role: "user" | "admin"
  status: "active" | "disabled"

  // OAuth
  googleId: String (sparse unique)
```

```

githubId: String (sparse unique)
provider: "local" | "google" | "github"
profilePicture: String
lastLogin: Date

// Password Reset
resetPasswordToken: String
resetPasswordExpires: Date

// Subscription (embedded)
subscription: {
  tier: "free" | "one-time" | "pro" | "premium" | "student" | "lifetime"
  plan: "monthly" | "yearly" | "3-months" | "lifetime" | "one-time"
  status: "active" | "cancelled" | "expired" | "trial" | "pending"
  startDate, endDate (indexed), receiptId, paymentId, orderId
  autoRenew: Boolean
  cancelledAt, cancelReason
}

// Usage Tracking (embedded)
usage: {
  resumesCreated, resumesThisMonth
  resumesDownloaded, resumesDownloadedThisMonth
  atsScans, atsScansThisMonth
  aiResumeExtractions, aiResumeExtractionsToday
  aiGenerationsUsed, aiGenerationsThisMonth
  tokensUsed
  lastResetDate, lastDailyReset
}

// Preferences
preferences: {
  currency: "INR" | "USD"
  notifications: { email, usageAlerts, renewalReminders }
}

```

Key instance methods:

- `comparePassword()` — bcrypt verify
- `hasActiveSubscription()` — check tier + expiry
- `isPremiumUser()` — paid tier check
- `canAccessFeature(feature)` — feature gate
- `getUsageLimit(limitType)` — tier-specific limits
- `hasReachedLimit(limitType)` — quota check
- `incrementUsage(type)` — usage tracking
- `resetMonthlyUsage()` / `resetDailyUsage()` — scheduled resets
- `checkSubscriptionExpiry()` — auto-downgrade expired subs

4.3 Resume Schema


```
Resume {
  userId: ObjectId (ref: User)
  resumeTitle: String
  description: String

  // Personal Info
  name: String (required)
  contact: { phone, email, linkedin, github, portfolio, location }

  // Sections
  summary: String
  skills: [{ category: String, items: [String] }]
  experience: [{
    company, title, location, startDate, endDate
    current: Boolean
    bullets: [String]
  }]
  education: [{
    institution, degree, field, location
    startDate, endDate, gpa
    bullets: [String]
  }]
  projects: [{
    name, description
    technologies: [String]
    link, bullets: [String]
  }]
  certifications: [{ name, issuer, date, credentialId, link }]
  achievements: [String]
  customSections: [{ id, title, items: [String] }]

  // Metadata
  rawText: String
  templateId: String (default: "classic")
  colorTheme: String

  // Subscription Tracking
  subscriptionInfo: {
    subscriptionId: ObjectId (ref: Subscription)
    createdWithTier: String
    createdWithSubscription: Boolean
    linkedAt: Date
  }
}
```

4.4 InterviewSession Schema

```
InterviewSession {
  userId: ObjectId (ref: User, indexed)
```

```
// Config
interviewType: "resume-based" | "job-description" | "technical" |
"behavioral" | "mixed"
role: String
experienceLevel: "fresher" | "junior" | "mid" | "senior" | "lead"
mode: "text" | "voice" | "mixed" | "live"

// Source
resumeId: ObjectId (ref: Resume)
resumeText: String
jobDescription: String
targetSkills: [String]

// State (indexed)
status: "created" | "in-progress" | "paused" | "completed" | "abandoned"
currentQuestionIndex: Number
totalQuestions: Number (5-20, default: 10)

// Questions Array
questions: [{
  questionNumber: Number
  questionText: String
  questionType: "technical" | "behavioral" | "situational" | "resume-
based" | "follow-up"
  category: String
  difficulty: "easy" | "medium" | "hard"

  // Answer
  userAnswer: String
  audioTranscription: String
  skipped: Boolean
  timeSpent: Number (seconds)
  answeredAt: Date

  // Evaluation (5 dimensions, each 0-10)
  evaluation: {
    score: Number (0-100)
    relevance: Number
    clarity: Number
    confidence: Number
    technical: Number
    roleFit: Number
    strengths: [String]
    improvements: [String]
    idealAnswer: String
    followUpQuestions: [String]
  }
}]

// Timing
startedAt: Date
completedAt: Date
totalTime: Number
```

```
// Results Reference
result: ObjectId (ref: InterviewResult)
}
```

4.5 InterviewResult Schema

```
InterviewResult {
  sessionId: ObjectId (ref: InterviewSession, unique)
  userId: ObjectId (ref: User, indexed)

  // Aggregate Scores (0-100)
  overallScore: Number
  dimensionScores: {
    technical: Number
    communication: Number
    problemSolving: Number
    culturalFit: Number
    confidence: Number
  }

  // Detailed Analysis
  strengths: [String]
  weaknesses: [String]
  keyInsights: [String]
  recommendations: [String]

  // Performance
  questionPerformance: [{
    questionNumber, questionText, score, keyPoints: [String]
  }]

  // AI Narrative
  executiveSummary: String
  detailedFeedback: String
  improvementPlan: [{ area, priority, suggestions: [String] }]
  nextSteps: [String]

  // Metadata
  hiringRecommendation: "strong-hire" | "hire" | "maybe" | "no-hire"
  generatedAt: Date
}
```

4.6 Other Models

```
// Subscription – payment history
Subscription {
  userId: ObjectId (ref: User)
  plan, tier, status
  amount, currency
}
```

```
    razorpayOrderId, razorpayPaymentId, razorpaySignature
    startDate, endDate
    features: [String]
  }

// Template – resume template definitions
Template {
  name, displayName, description
  category, tags: [String]
  thumbnail: String
  isActive: Boolean
  isPremium: Boolean
  config: Object (layout settings)
}

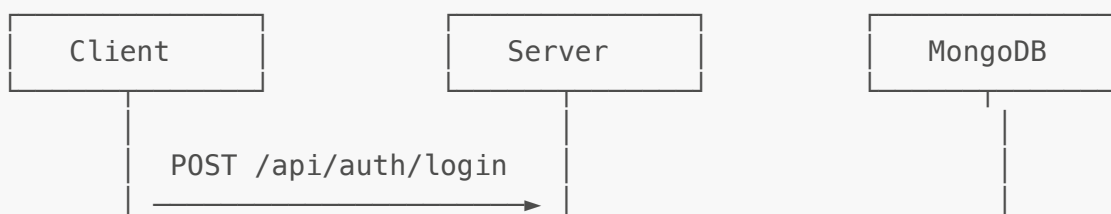
// AIUsage – per-request AI usage tracking
AIUsage {
  userId: ObjectId (ref: User)
  action: String
  model: String
  tokensUsed: Number
  cost: Number
  metadata: Object
  createdAt: Date (TTL indexed)
}

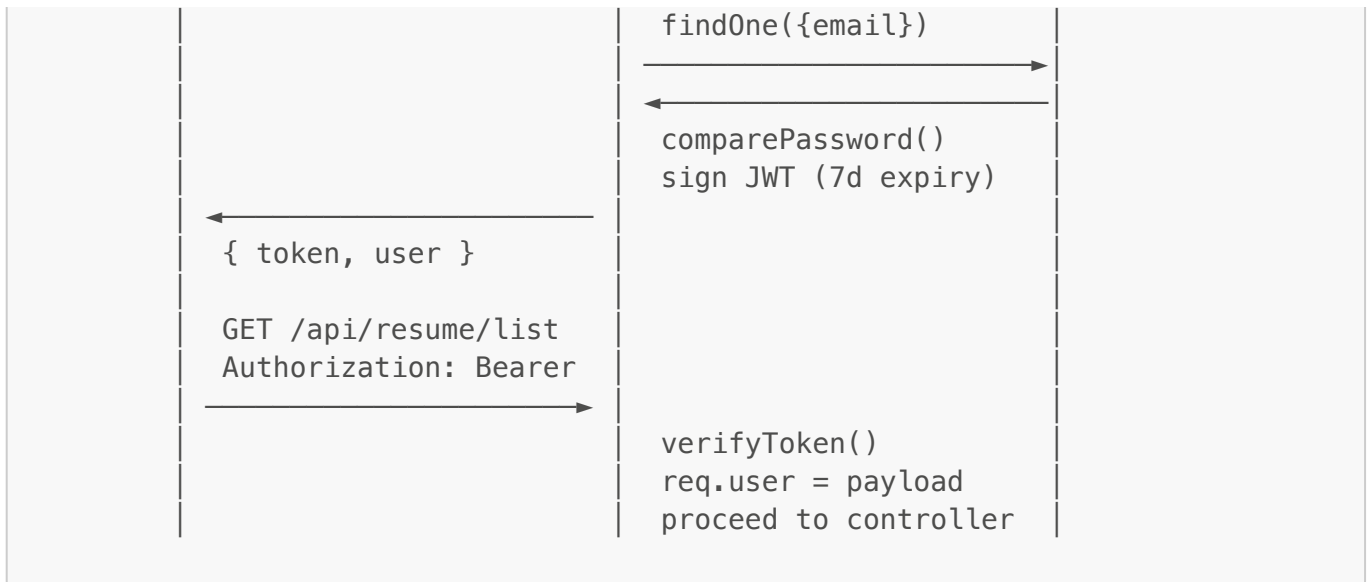
// AdminLog – audit trail
AdminLog {
  adminId: ObjectId (ref: User)
  action: String
  targetId: ObjectId
  targetType: String
  details: Object
  ip: String
  userAgent: String
  timestamp: Date
}

// Feedback, Contact – user communication
```

5. Authentication & Authorization

5.1 Auth Flow





5.2 OAuth Flow (Google/GitHub)

```

Client → GET /api/auth/google
      → Passport redirects to Google
      → Google redirects to /api/auth/google/callback
      → Passport exchanges code for profile
      → Find or create User (googleId lookup)
      → Sign JWT
      → Redirect to client /auth/callback?token=...
      → Client stores token in localStorage
  
```

5.3 Authorization Layers

```

Request
|
▼
authenticateToken (middleware)
├── Verify JWT signature
├── Attach req.user
└── Next()
    |
    ▼
    checkSubscription (middleware)
    ├── Reload user from DB
    ├── checkSubscriptionExpiry() – auto-downgrade if expired
    └── Attach fresh subscription data
        |
        ▼
        checkUsageLimit("featureName") (middleware)
        ├── hasReachedLimit(limitType) on User model
        └── 429 if over limit, Next() if ok
            |
            ▼
            checkAIQuota (middleware – AI routes only)
  
```

The diagram shows the authorization layers in a sequential flow. It starts with a Request, which goes through the authenticateToken middleware (verifying the JWT signature, attaching req.user, and calling Next()). This is followed by the checkSubscription middleware (reloading the user from the DB, checking subscription expiry, and attaching fresh subscription data). Next is the checkUsageLimit middleware (checking if the user has reached a limit for a specific feature and returning 429 if over the limit). Finally, the checkAIQuota middleware (for AI routes only) is executed.

```

├─ Check global AI quota in SystemSettings
├─ 503 if depleted
  │
  ▼
Controller

```

5.4 Role-Based Access

- **user** — standard access with tier-based feature gates
- **admin** — full access to `/api/admin/*` routes via `isAdmin` middleware

6. Core Feature Flows

6.1 Resume Creation Flow

1. User uploads PDF/DOCX
 POST `/api/resume/upload`
 → Multer parses file
 → pdf-parse / mammoth extracts text
 → Gemini AI structures data (name, experience, skills, etc.)
 → Returns structured JSON to client
2. User edits in Editor (`/editor/:id?`)
 → TipTap rich text editor per section
 → Real-time preview with selected template
 → Memoized ATS score calculation in `Editor.jsx`
 → Auto-save / manual save to POST `/api/resume/save`
3. AI Enhancement (per section)
 POST `/api/resume/enhance`
 → Gemini AI rewrites section with ATS keywords
 → Returns enhanced text
 → Client replaces section content
4. Download
 → Client renders template to PDF (`html2canvas` / `jsPDF`)
 → POST `/api/resume/track-download` (usage tracking)

6.2 ATS Analysis Flow

```

User submits resume + job description
  │
  ▼
POST /api/ats/analyze-resume
  │
  ▼
ats.controller.js → ats.service.js
  │

```

```

▼
Gemini AI analyzes:
- Keyword match %
- Missing keywords
- Skills alignment
- Section completeness
- Formatting quality
- ATS compatibility score (0-100)
|
▼
Returns structured report:
{ score, keywords, missing, suggestions, sections }

```

6.3 AI Interview Flow

```

Phase 1: Configuration
  User sets: type, role, level, mode, questions count
  Optionally links resume or pastes job description
  |
  ▼
Phase 2: Session Creation
  POST /api/interview/sessions
  → InterviewSession created in MongoDB (status: "created")
  → GPT-4o generates N questions based on config
  → Questions stored in session.questions[]
  |
  ▼
Phase 3: Interview Loop (per question)
  POST /api/interview/sessions/:id/start → Q1

  For TEXT mode:
    User types answer → POST /sessions/:id/answer
    GPT-4o evaluates (5 dimensions, 0-10 each)
    Returns score + feedback + next question

  For VOICE mode:
    Chatterbox TTS reads question aloud (audio stream)
    User records answer (MediaRecorder API)
    Audio blob → POST /sessions/:id/voice-answer (multipart)
    Server → Whisper STT service (port 5001)
    Transcription returned → GPT-4o evaluation
    Chatterbox TTS reads feedback aloud
    |
    ▼
Phase 4: Completion
  POST /api/interview/sessions/:id/complete
  → GPT-4o generates comprehensive InterviewResult
  → Result saved to MongoDB
  → Status → "completed"
  |
  ▼

```

Phase 5: Results

GET /api/interview/results/:sessionId

→ Full report with scores, insights, recommendations

→ Displayed in /interview-result/:sessionId

7. Voice Pipeline

7.1 TTS (Text-to-Speech) Priority Chain

```
server receives POST /api/voice/tts/synthesize
  |
  ▼
chatterbox.service.js
  → POST http://localhost:5002/synthesize
    |
    ├── SUCCESS → Return audio buffer to client
    |
    └── FAIL (service down)
        |
        ▼
      Return { fallback: true, text: "..."}
        |
        ▼
      Client uses Browser Web Speech API
      (window.speechSynthesis.speak())
```

7.2 STT (Speech-to-Text) Flow

```
Client records audio (MediaRecorder API)
  → webm/ogg blob
  → FormData { audio: blob }
  → POST /api/interview/sessions/:id/voice-answer
    |
    ▼
  Server proxies to voice-service (port 5001)
  POST http://localhost:5001/transcribe
    |
    ▼
  Python Flask → OpenAI Whisper model
  → Returns { transcription: String }
    |
    ▼
  Server uses transcription as userAnswer
  → GPT-4o evaluation proceeds
```

7.3 Disabled Services

- **ElevenLabs:** Fully disabled (payment issues). All ElevenLabs routes return 503.
- Routes affected: `/tts/synthesize-json`, `/tts/stream`, `/tts/test`, `/tts/voices`

8. Payment Flow

8.1 Razorpay Integration

```

Client: User selects plan on /pricing
      ↓
POST /api/subscription/create-order
  { plan: "pro", duration: "monthly" }
      ↓
server: razorpay.orders.create({ amount, currency: "INR" })
      → Returns { orderId, amount, currency }
      ↓
Client: Opens Razorpay checkout modal
      → User completes payment
      → Razorpay returns { paymentId, orderId, signature }
      ↓
POST /api/subscription/verify-payment
  { paymentId, orderId, signature }
      ↓
server: crypto.createHmac("sha256", secret)
        .update(`${orderId}|${paymentId}`)
        .digest("hex") === signature
      |
      | — VALID → Create Subscription record
      |               Update user.subscription (tier, dates)
      |               Return { success: true }
      |
      | — INVALID → 400 Payment verification failed
  
```

Also: POST /api/subscription/webhook (Razorpay server-side events)
 → `express.raw()` middleware (raw body for signature verification)
 → Handle: `payment.captured`, `subscription.cancelled`, etc.

8.2 Subscription Tiers & Feature Gates

```

Tier: free
  resumesPerMonth: 2
  aiGenerationsPerMonth: 5
  aiResumeExtractionsPerDay: 1
  resumeDownloadsPerMonth: 3
  atsScansPerMonth: 3
  
```

```
interviewsPerMonth: 2

Tier: pro
  resumesPerMonth: 20
  aiGenerationsPerMonth: 100
  aiResumeExtractionsPerDay: 5
  resumeDownloadsPerMonth: 50
  atsScansPerMonth: 30
  interviewsPerMonth: 20

Tier: premium / lifetime
  All limits significantly higher or unlimited
```

9. AI Integration Layer

9.1 Google Gemini (Resume Features)

Service: `server/services/gemini.service.js`

Used for:

- Resume text extraction & structuring from PDF/DOCX
- Section enhancement (rewriting with ATS keywords)
- Summary generation
- Skills categorization
- Achievement segregation
- ATS score analysis
- Job match scoring

Model: `gemini-1.5-flash` or `gemini-1.5-pro` (configurable per operation)

9.2 OpenAI GPT-4o (Interview Features)

Service: `server/services/openai.service.js` / `server/services/interview.service.js`

Used for:

- Interview question generation (temperature: 0.7)
- Answer evaluation (5 dimensions: relevance, clarity, confidence, technical, role-fit)
- Comprehensive interview report generation

Evaluation Dimensions (each 0-10):

```
relevance    – How relevant the answer is to the question
clarity      – How clearly expressed
confidence   – Level of confidence demonstrated
technical    – Technical accuracy (for tech questions)
roleFit      – Alignment with role requirements

Aggregate → score (0-100) for each question
```

9.3 AI Quota System

```
SystemSettings (MongoDB)
  globalAIQuota: {
    dailyTokenLimit: Number
    tokensUsedToday: Number
    resetAt: Date
  }

checkAIQuota middleware:
  → Load SystemSettings
  → Check tokensUsedToday < dailyTokenLimit
  → 503 if depleted
  → After request: increment tokensUsedToday by actual usage
```

9.4 User AI Preference

Users can switch between Gemini and GPT for certain operations:

- `POST /api/subscription/ai-preference { provider: "gemini" | "openai" }`
- Stored in user profile, respected by service layer

10. Middleware Stack

10.1 Global Middleware (all routes)

```
express.json()           - JSON body parser
express.urlencoded()     - Form body parser
cors({ origin: CLIENT_ORIGIN })
helmet()                 - Security headers
express-session          - Passport session (OAuth)
passport.initialize()
passport.session()
```

10.2 Route-Level Middleware

```
// Rate Limiters (express-rate-limit)
authLimiter      - 10 req/15min on auth routes
aiLimiter        - 30 req/min on AI routes
uploadLimiter    - 5 req/hour on upload routes
contactLimiter   - 3 req/hour on contact routes
feedbackLimiter  - 5 req/hour on feedback routes
interviewLimiter - 10 req/hour on interview creation
adminLimiter     - 100 req/min on admin routes
```

```

// Auth
authenticateToken - JWT verification → req.user
isAdmin           - role === "admin" check
checkSubscription - load fresh subscription + expiry check

// AI Gates
checkUsageLimit(type) - per-feature usage quotas
checkAIQuota           - global daily token quota

// Validation
validateRegister      - Joi/express-validator schemas
validateLogin
validateResumeUpdate
validateResumeId

// Upload
audioUpload.single("audio") - Multer for voice answers
multer (PDF/DOCX)           - Resume file upload

// Interview
checkInterviewLimit - per-user interview session limits

// Admin
logAdminAction       - Create AdminLog entry for every admin op

```

10.3 Error Handling

- Global error handler in `server.js`
- Structured error responses: `{ success: false, message: String, code?: String }`
- 401 — Unauthorized (no/invalid token)
- 403 — Forbidden (insufficient role/subscription)
- 429 — Rate limit / usage limit exceeded
- 503 — AI service unavailable

11. Admin Architecture

11.1 Admin Panel Flow

```

AdminProtectedRoute (client)
  → Checks user.role === "admin" in JWT payload
  → Redirects non-admins

AdminLayout (client)
  → Sidebar navigation
  → Admin-specific header/footer

All admin API calls:
  authenticateToken → isAdmin → adminLimiter → logAdminAction → Controller

```

11.2 Admin Capabilities

Feature	Capability
User Management	View all users, enable/disable, change role, delete
AI Analytics	Token usage by user/day/model, cost tracking
AI Quota	Override per-user quotas, reset daily limits, change tier
AI Extraction	Track per-user resume extraction usage
Contact Management	View messages, update status (read/resolved), delete
Feedback	Moderate feedback, upvote/resolve, delete
System Settings	Feature flags, rate limit config, global AI quota
Logs	Full audit trail of all admin actions (AdminLog)
Templates	Activate/deactivate resume templates, mark premium
Earnings	Razorpay payment analytics, revenue by plan
Dashboard	System-wide stats (users, resumes, sessions, revenue)

12. External Services

Service	Purpose	Communication	Fallback
MongoDB Atlas	Primary database	Mongoose ODM	None
Google Gemini API	Resume AI features	@google/generative-ai SDK	None
OpenAI GPT-4o	Interview AI features	openai SDK	None
Razorpay	Payment processing	REST API + webhook	None
Google OAuth	Social login	Passport.js strategy	Local auth
GitHub OAuth	Social login	Passport.js strategy	Local auth
Whisper STT (port 5001)	Voice transcription	HTTP → Python Flask	None
Chatterbox TTS (port 5002)	AI voice synthesis	HTTP → Python Flask	Browser Web Speech API
Nodemailer	Password reset emails	SMTP (EMAIL_USER/PASS)	None

Service	Purpose	Communication	Fallback
Adzuna API	Job search (optional)	REST (client-side)	Hidden route

13. Rate Limiting & Quota System

13.1 Request Rate Limits (express-rate-limit)

Limiter	Window	Max Requests	Applied To
authLimiter	15 min	10	/auth/register, /login, /forgot-password
aiLimiter	1 min	30	All AI routes
uploadLimiter	1 hour	5	/resume/upload, /ats/analyze-resume
contactLimiter	1 hour	3	/contact
feedbackLimiter	1 hour	5	/feedback
interviewLimiter	1 hour	10	POST /interview/sessions
adminLimiter	1 min	100	All /admin/*

13.2 Business Quota Limits (Per User, Per Tier)

Enforced by `checkUsageLimit(type)` middleware reading `User.usage.*`:

Limit Type	Free	Pro	Premium
resumesPerMonth	2	20	Unlimited
resumeDownloadsPerMonth	3	50	Unlimited
atsScansPerMonth	3	30	Unlimited
aiGenerationsPerMonth	5	100	Unlimited
aiResumeExtractionsPerDay	1	5	10
interviewsPerMonth	2	20	Unlimited

13.3 Quota Reset Schedule

- **Monthly counters** reset on `lastResetDate` (tracked per user)
- **Daily counters** reset on `lastDailyReset` (tracked per user)
- Reset triggered lazily on each request by `checkSubscriptionExpiry()` / middleware
- Admin can manually reset via `/api/admin/ai-quota/users/:userId/reset-daily`

Known Architectural Issues

1. **Duplicate route** in `voice.routes.js`: `GET /tts/health` defined twice (line 55 & 141) — second one shadows the first
 2. **Dead ElevenLabs references**: Routes exist but service is undefined — potential crash on those endpoints
 3. **Monolithic interview component**: `AIInterview.jsx` is 3300+ lines — should be split into sub-components
 4. **No WebSocket**: Voice interview uses polling/HTTP, not real-time WebSocket
 5. **Interview timer**: Auto-complete lacks grace period (abrupt cutoff)
 6. See `VOICE_SERVICE_AUDIT.md` for comprehensive voice service bugs
-

Generated from codebase analysis — March 2026